PDS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 4 | AULA 05 -

DATADOG - OBSERVABILIDADE "FULL-STACK" (AIOPS, WATCHDOG, RUM, SYNTHETICS & SECURITY)

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SUMÁRIO

O QUE VEM POR AI? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 42
REFERÊNCIAS 43

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VEM POR AI?

Nesta aula, aprofundaremos o conceito de **Observabilidade Full-Stack**, entendendo como dados de frontend e backend se conectam para explicar a experiência real do usuário. Vamos explorar os mecanismos de **AIOps com Watchdog**, a correlação entre **RUM e APM**, e a análise unificada de traces distribuídos, permitindo identificar regressões, gargalos e impactos de performance de forma proativa e contextualizada.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

HANDS ON

Durante esta aula prática teremos contato direto com os principais fluxos operacionais de observabilidade Full-Stack no Datadog:

- Configuração e validação do **Watchdog** para detecção automática de anomalias;
- Análise prática de **Stories** (correlação de eventos, regressões e causa raiz);
- Implementação do **RUM SDK** em aplicações frontend integradas a backend .NET;
- Validação da **propagação de headers de tracing** entre navegador e servidor;
- Navegação e análise de **Flame Graphs unificados (Frontend → Backend → DB);**
- Correlação entre **Core Web Vitals** e latência do backend;
- Uso do RUM e Logs para **validação de impacto real no usuário.**

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog – Observabilidade "Full-Stack" – APMs, Watchdog, RUM, Synthetics & Security

SAIBA MAIS

Watchdog e AIOps para detecção automática de anomalias

A complexidade das arquiteturas de software modernas, baseadas em microsserviços, containers e funções serverless, gerou um volume de dados de observabilidade (métricas, logs e traces) que ultrapassa a capacidade de análise humana. A detecção de problemas não é mais uma questão de definir limiares estáticos, mas sim de identificar desvios sutis em padrões de comportamento que se manifestam em múltiplas fontes de dados simultaneamente.

É neste contexto que surge o AIOps (Artificial Intelligence for IT Operations), uma disciplina que aplica inteligência artificial e Machine Learning para automatizar e aprimorar as operações de TI. O Datadog Watchdog é a implementação central do AIOps na plataforma Datadog. Ele atua como um motor de inteligência artificial que consome 100% dos dados de observabilidade, aprendendo o comportamento normal do seu sistema e identificando automaticamente anomalias, regressões e a causa raiz provável dos problemas.

O Watchdog transforma o monitoramento de um estado reativo e baseado em regras (onde você precisa saber o que procurar) para um estado proativo e baseado em aprendizado (onde o sistema encontra o que você não sabia que estava procurando). Ele não substitui os monitores tradicionais, mas os complementa, atuando como um "engenheiro de observabilidade" virtual que está constantemente analisando cada pedaço de telemetria.

Em um ambiente Full-Stack, o Watchdog é crucial porque ele pode correlacionar problemas que se manifestam em diferentes camadas:

- **Frontend (RUM):** detecta um aumento anômalo na taxa de erros de JavaScript ou uma degradação no LCP (Largest Contentful Paint).
- **Backend (APM):** identifica um pico de latência p99 em um serviço .NET específico.
- **Infraestrutura:** nota um aumento súbito na taxa de retransmissão de pacotes em um host.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Analisando 100% das métricas, logs e traces automaticamente

A premissa fundamental do Datadog Watchdog é que a detecção de problemas não deve ser limitada a um subconjunto de dados ou a limiares pré-definidos. Em vez disso, o Watchdog opera como um motor de AIOps que ingere e analisa 100% do fluxo de telemetria (métricas, logs e traces) em tempo real, 24 horas por dia, 7 dias por semana. Essa análise onipresente é o que permite a detecção de anomalias que seriam perdidas por qualquer sistema de monitoramento baseado em amostragem ou regras estáticas.

O Watchdog não trata métricas, logs e traces como silos de dados separados; ele os unifica em um modelo de dados comum para realizar a correlação.

Análise de Métricas (Time-Series): aplica algoritmos de Machine Learning (ML) a todas as séries temporais de métricas (infraestrutura, APM, customizadas) para aprender o comportamento normal.

- **Baseline Dinâmico:** ele constrói um *baseline* dinâmico que considera a sazonalidade (diária, semanal) e a tendência de longo prazo. Por exemplo, ele sabe que o uso de CPU do cluster .NET é naturalmente maior durante o horário comercial e ignora esses picos, mas alerta se o uso for anômalamente alto *para aquele horário específico*.
- **Detecção de Anomalias:** utiliza modelos estatísticos para identificar quando o valor atual da métrica se desvia significativamente da faixa de comportamento esperado.

Análise de Logs (Log Patterns): analisa o fluxo de logs para identificar padrões e agrupamentos que indicam problemas.

- **Clusterização de Logs:** ele agrupa logs semelhantes em *patterns* (padrões) e monitora a frequência desses padrões. Um aumento súbito na frequência de um padrão de log de erro, mesmo que o erro em si não seja novo, é um sinal de alerta.
- **Novos Padrões de Erro:** o Watchdog é treinado para identificar o surgimento de um **novo cluster de logs de erro** que nunca foi visto antes. Isso é um indicador fortíssimo de um bug introduzido por um deploy recente.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog – Observabilidade "Full-Stack"

Análise de Traces (APM): se concentra em identificar regressões de performance e erros em serviços e endpoints.

- **Regressões de Latência:** ele monitora a latência p95 e p99 de todos os serviços e recursos (endpoints) .NET. Se a latência de um endpoint específico (POST /api/orders) aumentar em 20% após um deploy, o Watchdog detecta essa regressão e a correlaciona com o evento de deploy.
- **Anomalias de Erro:** detecta picos anômalos na taxa de erros de um serviço, que podem não ser grandes o suficiente para disparar um SLO, mas são estatisticamente significativos.

A capacidade do Watchdog de processar e correlacionar 100% dos dados de observabilidade o transforma em um sistema de alerta precoce, encontrando problemas que se escondem na complexidade e no volume de dados.

Interpretando uma "Story" do Watchdog

O resultado da análise do Watchdog não é um simples alerta de limiar, mas sim uma "Story" (História). Uma Story é uma agregação inteligente de múltiplos eventos anômalos correlacionados, apresentada em uma narrativa concisa e acionável. O objetivo da Story é fornecer ao engenheiro de plantão o contexto completo do incidente, eliminando a necessidade de vasculhar manualmente dashboards, logs e traces para entender o que está acontecendo.

A Story é o ápice do AIOps, pois ela transforma dados brutos em conhecimento operacional. Ela não apenas diz *o que* está errado, mas sugere *por que* e *onde* o problema começou.

Uma Story bem-sucedida do Watchdog é composta por três elementos principais:

O Título e o Resumo (O Sintoma): é gerado automaticamente e descreve o sintoma principal. O resumo explica o que foi detectado e o impacto.

- **Exemplo:** "Aumento Anômalo de Latência no Serviço ecommerce-api após Deploy."
- **Conteúdo:** o Watchdog detectou que a latência p95 do endpoint /api/orders aumentou em 40% nos últimos 15 minutos, afetando 15% dos usuários.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

A Linha do Tempo e a Correlação (O Contexto): esta é a parte mais valiosa da Story. O Watchdog correlaciona o sintoma principal com outros eventos e anomalias que ocorreram no mesmo período.

- **Eventos de Deploy:** o Watchdog automaticamente sobrepõe eventos de deploy (obtidos via CI/CD ou tags de versão do APM) na linha do tempo. Se o aumento da latência começou 2 minutos após o deploy da versão 2.1.0 do serviço .NET, essa correlação é destacada como a causa provável.
- **Anomalias Secundárias:** ele lista outras anomalias que ocorreram no mesmo contexto. Por exemplo:
  - "Aumento de 15% no uso de CPU do host prod-web-03."
  - "Aumento de 50% na taxa de erros de log System.TimeoutException."
  - "Degradação de 20% no LCP (Largest Contentful Paint) para usuários da região Sudeste."
- **Análise de Causa Raiz (Root Cause Analysis - RCA):** o Watchdog utiliza seus algoritmos para sugerir a causa raiz mais provável. Por exemplo, se a latência do serviço .NET aumentou e o Watchdog detectou que a latência do banco de dados (PostgreSQL) chamado por esse serviço também aumentou, ele sugere que o problema está no banco de dados.

Links de Ação (A Próxima Etapa): a Story fornece links diretos para as ferramentas de investigação do Datadog, permitindo que o engenheiro valide a hipótese do Watchdog.

- **Link para o Trace:** um link direto para um trace de exemplo que demonstra a alta latência.
- **Link para o Log Explorer:** uma query pré-preenchida para o Log Explorer, filtrando pelos logs de erro do serviço afetado no período do incidente.
- **Link para o Dashboard:** um link para o dashboard de performance do serviço .NET afetado.

A Story do Watchdog transforma o processo de investigação de um esforço de coleta de dados em uma validação de hipóteses, reduzindo drasticamente o MTTR (Mean Time to Resolution).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog – Observabilidade "Full-Stack"
APM, Watchdog, RUM, Synthetics & Logs

Tipos de anomalias detectadas

A força do Watchdog reside em sua capacidade de aplicar diferentes modelos de Machine Learning a diferentes tipos de dados, permitindo a detecção de uma ampla gama de anomalias que indicam problemas de performance, estabilidade ou segurança. Ele não se limita a um único tipo de desvio, mas sim a uma matriz de padrões que, quando violados, sinalizam que o sistema está se comportando de maneira inesperada.

Para um ambiente Full-Stack, o Watchdog se concentra em anomalias que afetam a experiência do usuário (frontend), a saúde da aplicação (backend) e a estabilidade da infraestrutura.

O Watchdog categoriza as anomalias que detecta com base na fonte de dados e no tipo de desvio.

Anomalias de Performance (APM e RUM): estas anomalias se concentram na velocidade e na taxa de sucesso das transações.

Tipo de Anomalia	Métrica Alvo	Impacto em .NET
Latência p95/p99	`trace.aspnetcore.request.duration`	Aumento inesperado no tempo de resposta para a maioria dos usuários, frequentemente causado por contenção de recursos (locks, thread pool starvation) ou queries lentas.
Taxa de Erros	`trace.aspnetcore.request.errors`	Aumento súbito na proporção de requisições que retornam status code 5xx, indicando falhas na aplicação ou em serviços dependentes.
Degradação de RUM	`view.largest_contentful_paint`, `view.first_contentful_paint`	Aumento anômalo no tempo de carregamento de páginas para usuários reais, muitas vezes correlacionado a um backend lento ou a um recurso de frontend pesado.

Figura 1 - Anomalias de Performance
Fonte: Elaborado pelo autor (2026)

Anomalias de Estabilidade (Logs e Erros): estas anomalias se concentram na frequência e no tipo de erros que a aplicação está gerando.

Tipo de Anomalia	Fonte de Dados	Impacto em .NET
Novo Cluster de Logs	Logs	Surgimento de um novo padrão de log de erro que nunca foi visto antes, quase sempre indicando um bug introduzido por um deploy.
Padrão de Erro Existente	Logs	Aumento anômalo na frequência de um erro conhecido (ex: `System.TimeoutException`), sugerindo que o problema subjacente se agravou.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Figura 2 – Anomalias de Estabilidade
Fonte: Elaborado pelo autor (2026)

Anomalias de Infraestrutura (Métricas de Host/Container): estas anomalias se concentram na saúde dos recursos subjacentes que suportam a aplicação .NET.

Tipo de Anomalia	Métrica Alvo	Impacto em .NET
Picos de CPU/IO	`system.cpu.usage`, `system.io.util`	Aumento anômalo no uso de CPU ou na utilização de disco/rede, que pode levar à saturação e, consequentemente, ao aumento da latência da aplicação.
Uso de Memória	`system.mem.used`, `process.mem.rss`	Aumento inesperado no consumo de memória, que pode indicar um *memory leak* no código .NET ou um aumento na pressão do Garbage Collector (GC).
Contenção de Threads	`dotnet.thread_pool.active_threads`	Anomalias na contagem de threads ativas ou bloqueadas, indicando *thread pool starvation* ou *deadlocks* que afetam a capacidade de processamento da aplicação.

Figura 3 - Anomalias de Infraestrutura
Fonte: Elaborado pelo autor (2026)

Descoberta automática de regressões e deploys problemáticos

Um dos momentos de maior risco para a estabilidade de uma aplicação é o deploy. A introdução de uma nova versão de código, mesmo que pequena, pode inadvertidamente causar uma regressão de performance ou um aumento na taxa de erros. O Watchdog atua como um guardião automatizado do pipeline de CI/CD, correlacionando eventos de deploy com mudanças subsequentes no comportamento da aplicação.

A detecção automática de regressões de performance e erros é uma funcionalidade crítica do AIOps, pois permite que as equipes de engenharia identifiquem e revertam deploys problemáticos em minutos, minimizando o impacto no usuário final.

O Datadog APM e o Watchdog dependem da **Unified Service Tagging** para realizar essa correlação. Quando um serviço é instrumentado, ele deve ser tagueado com service, env e, crucialmente, version. Quando um novo deploy ocorre, a tag DD_VERSION muda. O Watchdog monitora o comportamento da aplicação *antes* e *depois* dessa mudança de tag.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog – Observabilidade "Full-Stack" – APMs, Watchdog, RUM, Synthetics & Security

Detecção de Regressão de Performance: compara métricas de performance chave (como latência p95 e p99) da versão N com a versão N-1.

- **Algoritmo:** ele usa testes estatísticos (como o teste t de Student ou testes não paramétricos) para determinar se a diferença no desempenho entre as duas versões é estatisticamente significativa e não apenas uma flutuação aleatória.
- **Alerta:** se a latência p95 do endpoint específico na versão 2.5.1 for significativamente maior do que na versão 2.5.0, o Watchdog gera uma Story de **Regressão de Latência**.

Detecção de Regressão de Erros: monitora a taxa de erros e o surgimento de novos tipos de erros.

- **Novo Bug:** se um novo cluster de logs de erro for detectado logo após a mudança de versão, o Watchdog correlaciona o novo bug diretamente com o deploy.
- **Aumento de Erros Conhecidos:** se a taxa de um erro conhecido aumentar significativamente, ele também é sinalizado como uma regressão.

O Watchdog estende o conceito de **Deployment Tracking** do Datadog. Ao invés de apenas marcar o deploy no dashboard, ele ativamente compara o desempenho *antes* e *depois* do marcador, fornecendo uma análise de impacto quantificada. Isso é crucial para a automação, pois permite que ferramentas de CI/CD consumam o resultado da análise do Watchdog e tomem decisões automatizadas, como pausar o *rollout* ou iniciar um *rollback* automático.

Quando confiar no Watchdog e quando investigar manualmente

O Watchdog é uma ferramenta poderosa de AIOps, mas não é um substituto para o julgamento e a experiência do engenheiro. A observabilidade Full-Stack de alto nível exige uma sinergia entre a detecção automatizada do Watchdog e a investigação manual e contextualizada do engenheiro.

Saber quando confiar cegamente na Story do Watchdog e quando iniciar uma investigação manual é crucial para evitar o "fadiga de alerta" e garantir que os recursos humanos sejam alocados de forma eficiente.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O Watchdog é altamente confiável em cenários onde a anomalia é clara e a correlação é forte.

Cenário	Justificativa	Ação Recomendada
Regressão Pós-Deploy	A correlação temporal com a mudança de versão é um indicador fortíssimo de causa e efeito.	**Confiança Alta.** Iniciar *rollback* ou correção imediata após a validação do trace de exemplo.
Anomalia de Log Pattern	O surgimento de um novo cluster de logs de erro é quase sempre um bug de código.	**Confiança Alta.** O Watchdog já agrupou os logs; o engenheiro deve ir diretamente para o Log Explorer para ver a *stack trace* completa.
Anomalia de Latência Simples	A latência p99 de um endpoint .NET está fora do *baseline* dinâmico, sem correlação aparente.	**Confiança Média.** O Watchdog identificou o sintoma. O engenheiro deve usar a Story para iniciar a investigação, focando no Trace Explorer para encontrar o gargalo.
Anomalia de Infraestrutura	Pico de CPU ou I/O que o Watchdog correlaciona com a latência do serviço.	**Confiança Alta.** O Watchdog forneceu a correlação entre camadas (infraestrutura → aplicação). O engenheiro deve investigar o host ou container específico.

Figura 4 – Anomalias
Fonte: Elaborado pelo autor (2026)

A investigação manual é necessária quando a Story do Watchdog é ambígua, a correlação é fraca ou o problema envolve lógica de negócio complexa.

Anomalias de Baixa Severidade e Longa Duração: se o Watchdog detectar uma anomalia sutil que persiste por dias, pode ser que o *baseline* tenha mudado ou que o problema seja um efeito colateral de uma mudança de tráfego (ex: um novo bot de rastreamento).

Correlação com Eventos Externos: o Watchdog não tem visibilidade sobre eventos externos (ex: um ataque DDoS que não é detectado pelo ASM, uma falha de provedor de terceiros). A Story pode apontar para uma anomalia de latência, mas o engenheiro deve verificar o status de serviços externos (ex: Stripe, AWS S3) que o serviço .NET consome.

Falsos Positivos: embora raro, o Watchdog pode gerar falsos positivos, especialmente em serviços com padrões de tráfego altamente irregulares ou após grandes eventos de marketing. O engenheiro deve validar a Story com o RUM para ver se usuários reais estão sendo afetados.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Regra de Ouro: o Watchdog fornece a hipótese e a evidência inicial. O engenheiro usa o contexto e o julgamento para confirmar a causa raiz e decidir a ação. O Watchdog economiza o tempo de *onde* procurar; o engenheiro decide *o que* fazer.

Conexão entre RUM e APM

A observabilidade Full-Stack exige uma visão unificada da experiência do usuário (Frontend) e da performance da aplicação (Backend). O Real User Monitoring (RUM) e o Application Performance Monitoring (APM) são as duas metades dessa equação. O RUM coleta dados de usuários reais (navegadores, dispositivos móveis), enquanto o APM monitora o código do lado do servidor (microsserviços .NET, bancos de dados).

A verdadeira inteligência da observabilidade Full-Stack reside na conexão entre esses dois domínios. O Datadog estabelece essa ponte através da propagação do contexto de tracing, permitindo que o engenheiro siga uma única requisição desde o clique do usuário no navegador até a execução do código no servidor e a consulta ao banco de dados.

Em arquiteturas de microsserviços, uma única ação do usuário pode desencadear dezenas de chamadas de API entre serviços. Sem a correlação RUM-APM, o engenheiro veria:

- **No RUM:** um usuário experimentou uma latência de 5 segundos.
- **No APM:** o serviço .NET teve um pico de latência de 5 segundos.

O engenheiro teria que adivinhar qual requisição do RUM corresponde a qual trace do APM. A conexão RUM-APM elimina essa adivinhação, injetando identificadores únicos de trace no tráfego do navegador.

A conexão RUM-APM permite:

- **Priorização Baseada no Usuário:** priorizar a otimização de endpoints .NET que estão causando a maior degradação na experiência do usuário final.
- **Diagnóstico de Latência:** determinar se a lentidão está no frontend (renderização, JavaScript) ou no backend (código .NET, banco de dados).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Flame Graphs Unificados:** visualizar o trace distribuído completo, que começa no navegador e atravessa todos os serviços de backend, em um único gráfico.

O “ponto cego” do backend: por que RUM é essencial

O Application Performance Monitoring (APM) é excelente para monitorar o que acontece no servidor. Ele mede a latência do código .NET, a performance das consultas ao banco de dados e a comunicação entre microsserviços. No entanto, o APM tem um ponto cego significativo: ele não tem visibilidade sobre o que acontece no lado do cliente (o navegador do usuário).

O APM só começa a medir o tempo de resposta quando a requisição HTTP chega ao servidor. Ele ignora:

- **Latência de Rede:** o tempo que a requisição leva para viajar do navegador do usuário até o servidor.
- **Performance do Frontend:** o tempo que o navegador leva para baixar, parsear e executar o JavaScript, renderizar o DOM e carregar recursos (imagens, CSS).
- **Erros de JavaScript:** falhas que ocorrem no navegador e que podem impedir o usuário de completar uma transação.

O **Real User Monitoring (RUM)** preenche esse ponto cego. O RUM SDK, injetado no frontend (ex: em uma aplicação Blazor ou React), coleta dados diretamente do navegador do usuário.

Métrica	Coletada pelo APM	Coletada pelo RUM
Tempo de Execução do Código .NET	Sim	Não
Tempo de Resposta do Servidor	Sim	Sim (como parte do tempo total)
Tempo de Download do JavaScript	Não	Sim
Largest Contentful Paint (LCP)	Não	Sim
Erros de JavaScript	Não	Sim

Figura 5 - Real User Monitoring
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

A lentidão percebida pelo usuário é a soma de todos os tempos: rede, servidor e frontend.

- **Cenário:** o APM mostra que o serviço .NET responde em 100ms. O RUM mostra que o usuário percebeu uma latência de 3 segundos.
- **Diagnóstico:** a lentidão não está no backend .NET. O problema pode ser a latência de rede (usuário em uma conexão lenta) ou a performance do frontend (JavaScript pesado ou renderização ineficiente).

O RUM é essencial porque ele fornece a **verdadeira métrica de performance**: a experiência do usuário. Ele permite que as equipes de engenharia .NET entendam o impacto real de suas otimizações (ou regressões) no usuário final, garantindo que o foco não seja apenas na performance do servidor, mas na performance Full-Stack.

Como o RUM injeta e propaga os cabeçalhos de tracing

A conexão entre o RUM e o APM é estabelecida através da propagação do contexto de tracing. O RUM SDK, rodando no navegador, é responsável por gerar o início de um trace distribuído e injetar os identificadores desse trace nos cabeçalhos HTTP de todas as requisições que o navegador faz para o backend.

Quando o serviço .NET recebe a requisição, o APM Tracer do .NET detecta esses cabeçalhos e usa os identificadores para continuar o trace, garantindo que o trace do backend seja o filho do trace do frontend.

O Datadog utiliza um conjunto de cabeçalhos HTTP para propagar o contexto de tracing. Os mais importantes são:

Cabeçalho	Função	Exemplo de Valor
`x-datadog-trace-id`	Identificador único do trace distribuído.	`1234567890123456789`
`x-datadog-parent-id`	Identificador do span pai (o último span no frontend).	`9876543210987654321`
`x-datadog-sampling-priority`	Indica se o trace deve ser amostrado (mantido) ou descartado.	`1` (Manter) ou `0` (Descartar)

Figura 6 - Cabeçalhos HTTP
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade Full-Stack: APM, Watchdog, RUM, Synthetics & Security

Quando o usuário clica em um botão que dispara uma requisição fetch ou XMLHttpRequest (XHR) para o backend .NET, o RUM SDK intercepta essa requisição antes que ela seja enviada e adiciona os cabeçalhos de tracing.

- **Geração:** o RUM gera um novo trace-id e um parent-id (que representa o span da ação do usuário no navegador).
- **Injeção:** os cabeçalhos são injetados na requisição HTTP.

O serviço .NET, instrumentado com o dd-trace-dotnet, recebe a requisição.

- **Detecção:** o Tracer do .NET (rodando como um *middleware* ou *hook*) detecta os cabeçalhos x-datadog-*.
- **Continuação:** em vez de iniciar um novo trace, o Tracer usa o trace-id e o parent-id recebidos para criar o span raiz do backend, garantindo que ele seja o filho do span do frontend.
- **Propagação Interna:** se o serviço .NET fizer chamadas para outros microsserviços ou bancos de dados, o Tracer do .NET propaga esses mesmos cabeçalhos para as requisições de saída, garantindo que o trace continue por toda a arquitetura.

O resultado final é um único trace distribuído que abrange toda a jornada da requisição:

Navegador (RUM) → Serviço .NET (APM) → Banco de Dados (APM)

Essa correlação automática é o pilar da observabilidade Full-Stack, permitindo que o engenheiro navegue do sintoma (latência no navegador) para a causa raiz (query lenta no banco de dados) em um único fluxo de trabalho.

Criando flame graphs unificados

O Flame Graph Unificado é a representação visual da correlação RUM-APM. Ele é uma ferramenta de diagnóstico que permite ao engenheiro visualizar a latência total de uma transação, desde o início no navegador (RUM) até a conclusão no banco de dados (APM), em um único gráfico hierárquico.

Este gráfico é fundamental para a observabilidade Full-Stack, pois elimina a ambiguidade sobre onde o tempo está sendo gasto.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O Span Raiz (Frontend - RUM): topo do gráfico é o span raiz, que representa a ação do usuário no navegador.

- **Serviço:** frontend-web (ou o nome do serviço RUM).
- **Duração:** o tempo total que o navegador levou para completar a ação (incluindo a latência de rede e o tempo de processamento do backend).

O Span do Backend (Serviço .NET - APM): próximo nível na hierarquia é o span do serviço .NET que recebeu a requisição.

- **Serviço:** ecommerce-api-dotnet.
- **Duração:** O tempo que o servidor .NET levou para processar a requisição.
- **Análise:** dentro deste span, o engenheiro pode ver os spans internos do .NET, como a execução do *Controller*, *Service* e *Repository*.

O Span de Dependência (Banco de Dados - APM): na profundidade do span do backend, o engenheiro encontrará o span de dependência, que geralmente é a chamada ao banco de dados.

- **Serviço:** sqlserver ou postgres.
- **Duração:** o tempo que a consulta SQL levou para ser executada.
- **Análise:** se este span for o mais largo do gráfico, a causa raiz da latência está na consulta ao banco de dados.

Core Web Vitals integrados ao APM

As Core Web Vitals (CWV) são um conjunto de métricas definidas pelo Google para quantificar a experiência do usuário em termos de carregamento, interatividade e estabilidade visual de uma página web. Elas são cruciais para o SEO e para a satisfação do usuário.

O Datadog RUM coleta e monitora as CWV, mas o valor Full-Stack reside na sua correlação direta com o APM.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Métrica	O que Mede	Limite "Bom"
Largest Contentful Paint (LCP)	Tempo que o maior elemento de conteúdo leva para carregar.	≤ 2.5 segundos
First Input Delay (FID)	Tempo entre a primeira interação do usuário e a resposta do navegador.	≤ 100 milissegundos
Cumulative Layout Shift (CLS)	Estabilidade visual da página (quanto o layout muda inesperadamente).	≤ 0.1

Figura 7 - Datadog RUM
Fonte: Elaborado pelo autor (2026)

A integração RUM-APM permite que o Datadog responda à pergunta: "A performance do meu backend .NET está afetando minhas Core Web Vitals?"

- **LCP:** é frequentemente afetado pelo tempo de resposta do servidor. Se o backend .NET demorar para retornar o HTML inicial ou os dados críticos, o LCP será alto. O Datadog permite correlacionar o LCP com a latência do endpoint .NET que serviu a página.
- **FID:** é primariamente um problema de frontend (JavaScript pesado bloqueando o *main thread*). No entanto, um backend lento pode indiretamente contribuir, pois o navegador pode estar ocupado processando a resposta lenta do servidor.
- **CLS:** é quase sempre um problema de frontend (imagens sem dimensões, injeção tardia de CSS). A correlação com o APM é menos direta, mas o trace unificado pode mostrar se a injeção de conteúdo que causa o CLS está ligada a uma resposta lenta do servidor.

Session Replay para análise de experiência do usuário

O Real User Monitoring (RUM) fornece a visão quantitativa da experiência do usuário: métricas (LCP, FID), contagens (erros, ações) e latências. No entanto, ele não responde à pergunta fundamental: "O que o usuário estava fazendo exatamente quando o problema ocorreu?"

O Session Replay do Datadog preenche essa lacuna, fornecendo a visão qualitativa. Ele é uma ferramenta que grava e reproduz a sessão real do usuário em um formato de vídeo, permitindo que engenheiros e gerentes de produto vejam a

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

jornada exata do usuário, incluindo movimentos do mouse, cliques, entradas de teclado e, crucialmente, o estado da interface no momento de um erro.

O Session Replay não é uma gravação de vídeo tradicional. Ele captura o DOM (Document Object Model) e os eventos de interação do usuário, recriando a experiência no lado do Datadog. Isso o torna leve e otimizado para a web.

Para aplicações complexas, como um portal de e-commerce ou um sistema de gestão construído em .NET (ex: Blazor ou ASP.NET Core com JavaScript pesado), um erro de JavaScript ou uma validação de formulário quebrada pode ser difícil de diagnosticar apenas com logs e traces.

- **Log de Erro:** uncaught TypeError: Cannot read properties of undefined (reading 'value')
- **Session Replay:** mostra que o usuário clicou em um botão antes que o formulário estivesse totalmente carregado, ou que um elemento de terceiros (como um widget de chat) estava sobrepondo o botão de envio.

O Session Replay transforma a investigação de um erro de frontend de uma análise de código abstrata para uma observação visual e contextualizada.

Como o Session Replay grava e reproduz a sessão real do usuário

O Session Replay não é uma gravação de vídeo tradicional (como um screencast), que seria pesada e inviável para monitoramento em escala. Em vez disso, ele utiliza uma técnica leve e eficiente baseada na captura de eventos e na reconstrução do DOM (Document Object Model).

O Datadog RUM SDK, instalado no frontend da aplicação .NET (ex: Blazor WebAssembly ou ASP.NET Core MVC/Razor Pages), atua como um gravador de eventos. Ele captura:

- **Mutações do DOM:** quaisquer alterações na estrutura da página (adição, remoção ou modificação de elementos).
- **Interações do Usuário:** cliques, movimentos do mouse, entradas de teclado, scrolls e mudanças de foco.
- **Eventos de Rede:** requisições XHR e fetch (que são cruciais para a correlação com o APM).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Eventos de Erro:** erros de JavaScript e falhas de recursos.

Esses eventos são serializados em um formato leve (JSON) e enviados ao Datadog em tempo real.

Quando um engenheiro visualiza um Session Replay no Datadog, o sistema executa um "player" que:

- **Recria o DOM Inicial:** o player carrega o estado inicial do DOM da página no momento em que a sessão começou.
- **Aplica as Mutações Sequencialmente:** ele reproduz a sequência exata de mutações do DOM e eventos de interação capturados.

O resultado é uma recriação fiel da experiência visual do usuário, como se o engenheiro estivesse olhando por cima do ombro do usuário.

O Session Replay é significativamente mais leve do que a gravação de vídeo, o que minimiza o impacto na performance do navegador do usuário. A gravação é baseada em dados, não em pixels.

Evento Capturado	Tipo de Dado	Tamanho Estimado
Carregamento Inicial da Página	Snapshot do DOM	~50-200 KB
Digitação do Nome	Eventos de `input` (apenas o valor final ou as mudanças)	~1 KB
Clique no Botão "Enviar"	Evento de `click` (coordenadas e elemento alvo)	< 1 KB
Resposta da API .NET	Evento de `fetch` (URL, status, duração)	< 1 KB

Figura 8 - Session Replay
Fonte: Elaborado pelo autor (2026)

O Session Replay é a ponte entre a experiência do usuário (o "o quê" e o "como") e a telemetria de backend (o "porquê"). Ele permite que o(a) engenheiro(a) veja exatamente o estado da UI quando um erro de JavaScript ocorreu, revelando problemas de CSS, *race conditions* ou interações inesperadas.

Casos de uso reais

Session Replay é mais do que uma ferramenta de depuração; é uma ferramenta de melhoria da Experiência do Usuário (UX). Ele transforma relatórios vagos de usuários ("O botão não funcionou") em evidências visuais concretas,

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

permitindo que as equipes de desenvolvimento .NET e frontend resolvam problemas que são difíceis de replicar em ambientes de teste.

Privacidade e máscara de dados

A capacidade do Session Replay de gravar a interação do usuário levanta questões críticas de privacidade e conformidade regulatória (como GDPR, LGPD e CCPA). É imperativo que dados sensíveis, como números de cartão de crédito, senhas, CPFs e informações de saúde, nunca sejam capturados e enviados para o Datadog.

O Datadog Session Replay aborda essa preocupação através de um mecanismo robusto de **máscara de dados** (data masking), que é configurado no momento da inicialização do RUM SDK.

O RUM SDK oferece o parâmetro defaultPrivacyLevel na inicialização, que define o nível de máscara aplicado a todos os elementos da página por padrão. Níveis de privacidade:

Nível	Descrição	Comportamento	Uso Recomendado
`mask` (Padrão)	Máscara de Conteúdo	O conteúdo de todos os elementos de texto é substituído por caracteres `x`. O formato (número de palavras, comprimento) é preservado.	Ambientes de Produção (Recomendado)
`mask-user-input`	Máscara de Entrada do Usuário	O conteúdo de todos os campos de entrada (`<input>`, `<textarea>`) é substituído por `x`. O conteúdo de outros elementos de texto é preservado.	Ambientes de Teste/Desenvolvimento
`allow`	Sem Máscara	Nenhum conteúdo é mascarado. **Não recomendado** para produção.	Debugging local ou ambientes controlados.

Figura 9 - RUM SDK
Fonte: Elaborado pelo autor (2026)

Exemplo de mascara (mask):

Conteúdo Original	Conteúdo Gravado
`Nome: João da Silva`	`Nome: xxxx xx xxxxx`
`CPF: 123.456.789-00`	`CPF: xxx.xxx.xxx-xx`
`Senha: *********`	`Senha: xxxxxxxxx`

Figura 10 – Exemplo de Mask

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Fonte: Elaborado pelo autor (2026)

Em alguns casos, pode ser necessário mascarar um elemento específico que não é um campo de entrada, ou, inversamente, permitir que o conteúdo de um elemento não sensível seja visível. Isso é feito através de atributos HTML específicos:

Ação	Atributo HTML	Exemplo
Mascarar Elemento	`data-dd-privacy="mask"`	`<div data-dd-privacy="mask">Este texto será mascarado.</div>`
Permitir Elemento	`data-dd-privacy="allow"`	`<div data-dd-privacy="allow">Este texto será visível.</div>`
Ignorar Elemento	`data-dd-privacy="hidden"`	O elemento é removido do DOM no replay.

Figura 11 – Atributo HTML
Fonte: Elaborado pelo autor (2026)

A implementação correta da máscara de dados é uma responsabilidade legal e ética do engenheiro. O Session Replay é uma ferramenta poderosa, mas deve ser usada com o máximo cuidado para garantir a conformidade e a confiança do usuário.

Navegando por uma sessão: timeline, eventos, erros e recursos

A interface do Session Replay no Datadog é projetada para ser um centro de comando para a análise da experiência do usuário. Ela combina a reprodução visual da sessão com um painel de telemetria sincronizado, permitindo que o engenheiro correlacione o que está vendo com os dados técnicos coletados pelo RUM.

A tela de Session Replay é dividida em três áreas principais:

O Player Visual (Centro)

- **Reprodução:** onde a sessão é recriada visualmente. O engenheiro pode pausar, avançar e retroceder a reprodução.
- **Velocidade:** a velocidade de reprodução pode ser ajustada (ex: 1x, 2x, 4x) para acelerar momentos de inatividade ou lentidão.
- **Indicadores:** o player destaca visualmente cliques, entradas de texto e movimentos do mouse.

A Timeline (Base): é a representação gráfica da duração da sessão, com marcadores visuais para eventos importantes.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack" (APM, Watchdog, RUM, Synthetics & Segurança)

- Marcadores de Eventos:
  - **Erros:** marcadores vermelhos indicam o momento exato em que um erro de JavaScript ou de recurso ocorreu.
  - **Ações:** marcadores azuis indicam interações do usuário (cliques, mudanças de rota).
  - **Frustração:** marcadores laranjas indicam sinais de frustração, como *rage clicks* (múltiplos cliques rápidos no mesmo lugar) ou *dead clicks* (cliques em elementos não interativos).
- **Navegação Rápida:** o engenheiro pode clicar diretamente em um marcador de erro na timeline para pular instantaneamente para o momento exato em que o problema ocorreu.

O Painel de Telemetria (Lateral): fornece o contexto técnico sincronizado com o momento atual da reprodução.

- **Eventos:** lista cronológica de todos os eventos RUM (erros, ações, recursos, logs) que ocorreram durante a sessão.
- **Erros:** detalhes de cada erro de JavaScript, incluindo a *stack trace* completa.
- **Recursos:** lista de todos os recursos carregados (imagens, CSS, JS), com seus tempos de carregamento e status de rede.
- **Atributos:** metadados da sessão, como ID do usuário, versão da aplicação (.NET version tag), ambiente (env), navegador e localização.

O uso combinado desses componentes permite que o engenheiro correlacione o comportamento do usuário com os eventos técnicos subjacentes.

Conexão direta com backend via “View APM Trace” no RUM

O Session Replay e o RUM fornecem o contexto visual e de performance do frontend. No entanto, a causa raiz de um problema de frontend é frequentemente um gargalo ou erro no backend. O recurso **"View APM Trace"** é o ponto de pivô que permite ao engenheiro saltar instantaneamente da experiência visual do usuário para a análise profunda do código .NET no servidor.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O fluxo de investigação ideal começa no frontend (RUM/Session Replay) e, quando necessário, pivota para o backend (APM) sem perder o contexto.

Identificação do Evento Crítico no Session Replay

O engenheiro está assistindo a um Session Replay e observa que o usuário clica em um botão que dispara uma chamada de API (POST /api/orders). O replay mostra que a tela fica em branco por 5 segundos antes de a resposta chegar.

Localização do Recurso no Painel de Telemetria

No painel de telemetria do Session Replay, o engenheiro localiza o evento de recurso correspondente à chamada POST /api/orders. Este evento de recurso terá um tempo de duração de 5 segundos.

O Pivô: "View APM Trace"

Ao lado do evento de recurso, o Datadog exibe um botão ou link chamado "View APM Trace". Este link só aparece se o RUM SDK tiver injetado com sucesso os cabeçalhos de tracing e o APM Tracer do .NET tiver recebido e processado a requisição.

Ao clicar neste link, o engenheiro é levado diretamente para o Flame Graph Unificado (T2.S3) no APM Trace Explorer.

O Valor da Continuidade

O recurso "View APM Trace" garante a continuidade da investigação. Ele elimina a necessidade de:

- Copiar e colar IDs de trace manualmente.
- Perder o contexto do tempo e do usuário.
- Alternar entre diferentes ferramentas e interfaces.

Ele transforma a observabilidade Full-Stack de um conceito teórico em um fluxo de trabalho prático e eficiente, permitindo que as equipes .NET resolvam problemas de ponta a ponta com a máxima velocidade.

Testes Sintéticos

O monitoramento de observabilidade (métricas, logs, traces, RUM) é, por natureza, **reativo**: ele informa sobre problemas que já estão afetando os usuários ou

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

que já ocorreram. O **Monitoramento Sintético (Synthetic Monitoring)** do Datadog inverte essa lógica, oferecendo uma abordagem **proativa**.

Testes Sintéticos são scripts automatizados que simulam o comportamento de um usuário ou de um sistema, executados em intervalos regulares a partir de localizações geográficas controladas. Eles agem como "robôs" que testam continuamente a disponibilidade e a performance de endpoints de API e fluxos de usuário em navegadores reais.

Para uma aplicação em produção, os Testes Sintéticos garantem que:

- **Disponibilidade 24/7:** a aplicação está sempre acessível a partir de diferentes regiões do mundo.
- **Performance Consistente:** a latência de endpoints críticos (ex: POST /api/login) permanece dentro dos limites aceitáveis.
- **Fluxos de Negócio Críticos:** o caminho de conversão (ex: Login → Adicionar ao Carrinho → Checkout) está funcional e rápido.

Ao detectar uma falha em um Teste Sintético, o Datadog alerta a equipe **antes** que um número significativo de usuários reais (monitorados pelo RUM) seja afetado. Além disso, a falha de um teste sintético fornece um **diagnóstico imediato** (o endpoint exato que falhou, o status code, o tempo de resposta), o que acelera o MTTR (Mean Time to Resolution).

Quando usar API Test vs Browser Test

O Datadog Synthetic Monitoring oferece dois tipos principais de testes, cada um focado em uma camada diferente da aplicação. A escolha entre um **API Test** e um **Browser Test** depende do objetivo do monitoramento

O API Test simula uma requisição HTTP direta a um endpoint de API. Ele opera na camada de rede e aplicação, sem renderizar o frontend.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Característica	Detalhe
Objetivo	Testar a disponibilidade, performance e lógica de negócio do backend (serviços .NET).
O que mede	Status code, tempo de resposta, cabeçalhos, conteúdo do payload (JSON/XML).
Vantagens	Rápido, leve, barato de executar, ideal para monitoramento de latência e lógica de negócio.
Desvantagens	Não detecta problemas de frontend (JavaScript, CSS, renderização).
Caso de Uso .NET	Monitorar a latência p99 do endpoint `POST /api/login` ou a integridade do endpoint `GET /api/health`.

Figura 12 – API Test
Fonte: Elaborado pelo autor (2026)

O Browser Test simula um usuário interagindo com a aplicação em um navegador real (Chromium ou Firefox). Ele renderiza a página e executa o JavaScript.

Característica	Detalhe
Objetivo	Testar a experiência do usuário final e a funcionalidade de fluxos de negócio críticos.
O que mede	Tempo de carregamento da página, Core Web Vitals (LCP, CLS), cliques, entradas de texto, erros de JavaScript.
Vantagens	Cobertura Full-Stack, detecta problemas de integração frontend-backend e erros de renderização.
Desvantagens	Mais lento, mais caro de executar, requer mais manutenção (seletor de elementos pode mudar).
Caso de Uso .NET	Simular o fluxo completo de "Login → Adicionar Produto → Checkout" para garantir que a jornada de conversão está funcional.

Figura 13 - Browser Test
Fonte: Elaborado pelo autor (2026)

Matriz de decisão, regra de ouro:

- Use **API Tests** para isolar e monitorar a performance e a lógica de negócio de endpoints individuais.
- Use **Browser Tests** para monitorar a experiência do usuário e a funcionalidade de fluxos de negócio de ponta a ponta.

Criação de API Tests com validações avançadas

Um API Test básico verifica se um endpoint retorna um Status Code 200 OK. No entanto, para aplicações .NET complexas, isso não é suficiente. Um endpoint pode retornar 200 mesmo que o payload esteja incompleto, incorreto ou tenha uma latência inaceitável.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack" - APMs, Watchdog, RUM, Synthetics e Segurança

O Datadog Synthetic Monitoring permite a criação de validações avançadas (Assertions) que garantem a integridade completa da resposta do backend. Alguns tipos de validações avançadas:

Validação de Status Code e Tempo de Resposta: estas são as validações fundamentais de performance e disponibilidade.

- **Status Code:** garantir que o código de resposta esteja dentro de uma faixa esperada (ex: 200 ou 201)
- **Tempo de Resposta:** definir um limite de latência. Para um endpoint crítico de .NET (ex: POST /api/auth), o limite pode ser Tempo de Resposta < 300ms.

Validação de Payload (JSON/XML): esta validação garante que o conteúdo da resposta do backend .NET está correto.

- **Validação de Conteúdo:** verificar se um valor específico está presente no corpo da resposta.
  - **Exemplo:** para GET /api/products/123, validar se o campo product_name contém o valor "Laptop X1".
- **Validação de JSON Path:** usar a sintaxe JSON Path para extrair e validar valores aninhados.
  - **Exemplo:** validar se $.inventory.stock_level é maior que 0.

Validação de Schema (Estrutura): esta é uma validação crucial para a integridade da API. Ela garante que a estrutura do JSON retornado pelo serviço .NET (o *schema*) não foi alterada por um deploy.

- **Exemplo:** validar se o campo product_id existe e é do tipo string, e se o campo price existe e é do tipo number.
- **Benefício:** um deploy que remove acidentalmente um campo essencial da resposta JSON será detectado imediatamente, antes que o frontend comece a falhar.

Validação de SSL e Headers

- **SSL:** garantir que o certificado SSL/TLS está válido e não expirou.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack" (APIs, Watchdog, RUM, Synthetics e Segurança)

- **Headers:** verificar se cabeçalhos de segurança (ex: Content-Security-Policy) ou cabeçalhos de negócio (ex: X-Service-Version) estão presentes e com o valor correto.

A capacidade de realizar validações avançadas transforma o API Test de um simples *ping* de disponibilidade em um poderoso teste de regressão funcional e de contrato de API.

Multistep API Tests para fluxos com autenticação/token

A maioria das APIs de produção, especialmente aquelas que suportam aplicações .NET, requer **autenticação** (ex: OAuth 2.0, JWT). Um teste de API simples não pode acessar endpoints protegidos. O **Multistep API Test** (Teste de API de Múltiplas Etapas) do Datadog resolve isso, permitindo simular um fluxo de negócio completo que envolve a obtenção de um token e seu uso em requisições subsequentes.

O segredo do Multistep API Test é a capacidade de **extrair** dados da resposta de uma etapa e armazená-los em uma **variável** que pode ser usada na próxima etapa.

Etapa 1 - autenticação (Obter o Token): primeiro passo em um fluxo autenticado é sempre obter um token de acesso.

- **Ação:** POST para o endpoint de login (/api/auth/token) com credenciais de teste (usuário/senha).
- **Extração:** o teste deve validar o Status Code 200 e, em seguida, usar uma regra de extração (ex: JSON Path) para capturar o valor do campo access_token da resposta.
- **Variável:** o valor extraído é armazenado em uma variável de teste (ex: {{AUTH_TOKEN}}).

Etapa 2 - Uso do Token (Acesso ao Recurso .NET): token extraído é então usado para acessar um endpoint protegido.

- **Ação:** GET para um endpoint protegido (ex: /api/user/profile).
- **Injeção:** o teste injeta a variável {{AUTH_TOKEN}} no cabeçalho Authorization da requisição.
    - **Header:** Authorization: Bearer {{AUTH_TOKEN}}.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Validação:** o teste valida o Status Code 200 e o conteúdo do perfil do usuário.

Este teste de múltiplas etapas garante que a lógica de autenticação, a criação de recursos e a transição de estado estão funcionando corretamente e dentro dos limites de latência aceitáveis. Se a Etapa 1 falhar, as etapas subsequentes não serão executadas, e o alerta será disparado com o diagnóstico preciso da falha de login.

O Multistep API Test é essencial para monitorar a saúde dos fluxos de negócio críticos que dependem da correta gestão de estado e autenticação.

Browser Tests com gravação automática

O Browser Test é a forma mais fiel de simular a experiência do usuário final. No passado, a criação de testes de navegador (como Selenium) era complexa e exigia conhecimento de programação. O Datadog simplifica drasticamente esse processo com a Gravação Automática (Recorder).

O Recorder é uma extensão de navegador que permite ao engenheiro interagir com a aplicação normalmente, enquanto o Datadog registra cada clique, entrada de texto e navegação, traduzindo essas ações em um script de teste sintético.

O processo de gravação:

- **Instalação:** o engenheiro instala a extensão do Datadog Synthetic Recorder no navegador.
- **Gravação:** o engenheiro inicia a gravação e executa o fluxo de negócio que deseja monitorar (ex: navegar até a página de produto, adicionar ao carrinho, ir para o checkout).
- **Geração do Script:** ao finalizar a gravação, o Recorder gera um script de teste que contém:
  - **Ações:** click, typeText, selectOption, scroll.
  - **Seletores:** o Datadog usa seletores robustos (ex: data-testid ou id) para garantir que o teste não quebre facilmente com pequenas mudanças de layout.
  - **Validações:** o Recorder sugere validações automáticas (ex: "O texto 'Seu carrinho está vazio' não deve estar visível").

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack" APM, Watchdog, RUM, Synthetics & Security

Vantagens da gravação automática

- **Acessibilidade:** permite que engenheiros de QA, SREs e até mesmo gerentes de produto criem testes complexos sem escrever código.
- **Fidelidade:** o teste é executado em um navegador real, simulando com precisão o ambiente do usuário.
- **Medição de CWV:** o Browser Test mede automaticamente as Core Web Vitals (LCP, CLS) do fluxo, fornecendo uma métrica proativa da experiência do usuário.

O Browser Test, quando executado, gera um trace distribuído completo, exatamente como um usuário real.

- **RUM:** o teste sintético aparece no RUM Explorer como uma sessão sintética, permitindo que o engenheiro veja o Session Replay do robô e o flame graph unificado.
- **APM:** se o teste falhar devido a um erro de backend .NET, o trace do APM correlacionado estará disponível, permitindo que o engenheiro pule diretamente para o código que causou a falha.

O Browser Test com gravação automática é a ferramenta ideal para garantir que a funcionalidade e a performance da interface do usuário estejam sempre operacionais, atuando como a primeira linha de defesa contra regressões de frontend.

Alertas 24/7: cobertura proativa contra deploys problemáticos

A principal função do Synthetic Monitoring é fornecer uma cobertura de alerta 24 horas por dia, 7 dias por semana, para os fluxos de negócio mais críticos. Ao contrário dos testes de QA que são executados apenas durante o pipeline de CI/CD, os Testes Sintéticos são executados continuamente em produção.

Essa execução contínua atua como um sistema de alerta precoce, detectando problemas de disponibilidade e performance que podem surgir a qualquer momento, seja por um deploy, uma falha de infraestrutura ou um problema de terceiros.

O Datadog permite criar monitores que disparam alertas com base nos resultados dos Testes Sintéticos.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Alerta de Falha (Disponibilidade)

O monitor mais comum é o de falha. Ele dispara um alerta se o teste sintético falhar em um número X de localizações ou em um período Y de tempo.

- **Configuração:** "Alertar se o Browser Test: Checkout Flow falhar em **2** localizações geográficas por **5** minutos."
- **Vantagem:** se o teste falhar em apenas uma localização, pode ser um problema de rede regional. Se falhar em duas ou mais, é um problema na aplicação.

Alerta de Regressão de Performance (Latência)

O monitor de performance dispara um alerta se o tempo de execução do teste exceder um limite definido.

- **Configuração:** "Alertar se o API Test: POST /api/login tiver um tempo de resposta superior a **500ms** em **3** execuções consecutivas."
- **Vantagem:** garante que a performance do backend .NET não se degrade silenciosamente.

Integração com CI/CD (Deployment Gates)

O uso mais avançado de Testes Sintéticos é como Deployment Gates no pipeline de CI/CD.

- **Mecanismo:** após o deploy para um ambiente de *staging* ou *canary*, o pipeline executa um conjunto de Testes Sintéticos críticos.
- **Decisão:** se todos os testes passarem, o pipeline continua para o próximo estágio (ex: *rollout* completo). Se um teste falhar, o pipeline é interrompido e um *rollback* pode ser iniciado automaticamente.

Isso transforma o Teste Sintético de uma ferramenta de monitoramento em uma ferramenta de **qualidade de software**, garantindo que apenas código funcional e performático chegue à produção.

App and API Protection (AAP)

A segurança de aplicações é um campo em constante evolução, e as ferramentas tradicionais muitas vezes lutam para acompanhar a velocidade e a

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack" (APM, Watchdog, RUM, Synthetics, Security)

complexidade das arquiteturas modernas de microsserviços. O Datadog reconheceu essa lacuna e evoluiu sua oferta de segurança, substituindo o antigo Application Security Management (ASM) pelo App and API Protection (AAP).

O AAP representa uma abordagem mais abrangente e integrada à segurança, focada em proteger aplicações e APIs contra uma vasta gama de ataques, desde as vulnerabilidades mais comuns (como as do OWASP Top 10) até abusos de API e ataques à lógica de negócio. Sua principal inovação reside na profunda integração com a plataforma de observabilidade do Datadog, especialmente o APM (Application Performance Monitoring).

Ao invés de ser uma solução de segurança separada, o AAP é uma extensão do APM. Ele utiliza os mesmos agentes e bibliotecas de instrumentação que coletam métricas de performance e traces distribuídos para inspecionar o tráfego em tempo real. Isso significa que a segurança não é uma camada externa adicionada, mas uma capacidade inerente à observabilidade da aplicação.

Vantagens do AAP sobre Abordagens Tradicionais:

Contexto de Execução: o AAP opera dentro do contexto da aplicação, tendo visibilidade sobre o código .NET em execução, as dependências e o estado da requisição. Isso permite uma detecção de ameaças mais precisa e com menos falsos positivos do que um WAF (Web Application Firewall) de rede, que vê apenas o tráfego bruto.

Correlação com Observabilidade: eventos de segurança detectados pelo AAP são automaticamente correlacionados com os traces do APM, logs e métricas. Isso fornece uma visão Full-Stack do ataque, desde a requisição maliciosa até o impacto no código .NET e nos recursos da aplicação.

Proteção em Tempo Real: o AAP pode ser configurado para bloquear ataques em tempo real, impedindo que requisições maliciosas atinjam o código vulnerável ou o banco de dados.

Para aplicações .NET, o AAP se integra perfeitamente com o Datadog .NET Tracer. Ele estende as capacidades de monitoramento de performance para incluir a detecção e proteção contra ameaças, sem a necessidade de instrumentação adicional complexa. Isso garante que as aplicações ASP.NET Core, .NET Framework e outras baseadas em .NET estejam protegidas desde o desenvolvimento até a produção.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack" (APM, Watchdog, RUM, Synthetics & Security)

Como o AAP Usa o APM

O App and API Protection (AAP) do Datadog não é uma ferramenta de segurança isolada; ele é um módulo avançado que se integra diretamente ao APM Tracer. O Tracer, que já é responsável por coletar dados de performance e gerar traces distribuídos para aplicações .NET, é estendido para atuar como um sensor de segurança em tempo real. Essa abordagem permite que o AAP inspecione o tráfego e o comportamento da aplicação com um nível de contexto e precisão que soluções de segurança tradicionais não conseguem.

Instrumentação e Captura de Dados (Tracer do .NET): quando uma requisição HTTP chega a um serviço .NET instrumentado com o dd-trace-dotnet (o Tracer do Datadog para .NET), o Tracer realiza as seguintes ações:

- Geração de Trace: inicia ou continua um trace distribuído, criando o span raiz para a requisição.
- Captura de Contexto: captura automaticamente o corpo da requisição (payload), os cabeçalhos HTTP, os parâmetros de URL e outras informações relevantes da requisição de entrada.
- Inspeção de Segurança: o módulo AAP, integrado ao Tracer, inspeciona esses dados de entrada em busca de padrões de ataque conhecidos, anomalias e comportamentos suspeitos. Essa inspeção ocorre in-process, ou seja, dentro do próprio processo da aplicação, antes que a requisição seja totalmente processada pelo código da aplicação.

Análise de Ameaças (AAP Engine): aplica um conjunto de regras de detecção de ameaças (Threat Detection Rules) e modelos de Machine Learning aos dados capturados. Essas regras são continuamente atualizadas pelo Datadog Security Labs e cobrem uma vasta gama de ataques, incluindo:

- Vulnerabilidades OWASP Top 10: Injeção SQL, XSS, SSRF, etc.
- Abuso de API: tentativas de exploração de lógica de negócio, enumeração de usuários, etc.
- Ataques de Dia Zero: detecção de padrões anômalos que podem indicar ataques ainda não catalogados.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack" (APM, Watchdog, RUM, Synthetics & Security)

Enriquecimento do Trace e Ação (Datadog Platform): se uma ameaça é detectada, o AAP enriquece o span do APM com informações de segurança e pode tomar ações configuradas:

- Tags de Segurança: tags específicas (ex: appsec.event:true, appsec.event.rule.id, appsec.event.rule.name, appsec.attack.type) são adicionadas ao span do APM, tornando o evento de segurança parte integrante do trace distribuído.
- Eventos de Segurança: um evento de segurança detalhado é gerado e enviado para o Datadog Security Explorer e Security Monitoring.
- Bloqueio (Opcional): se configurado no modo de bloqueio, o AAP pode interromper a execução da requisição maliciosa e retornar uma resposta de erro (ex: 403 Forbidden) diretamente, impedindo que o ataque atinja o código da aplicação ou o banco de dados. Isso é conhecido como Runtime Application Self-Protection (RASP).

A integração profunda do AAP com o APM é sua maior força. Ao correlacionar eventos de segurança com traces de performance, o AAP fornece um contexto inigualável para a investigação de incidentes:

- Visibilidade Full-Stack: o engenheiro pode ver a jornada completa da requisição, desde o cliente até o código .NET, e identificar exatamente onde e como o ataque ocorreu.
- Diagnóstico Preciso: o trace inclui o payload do ataque, o endpoint alvo, o usuário (se autenticado), a versão do serviço e a stack trace da execução, acelerando drasticamente a identificação da causa raiz e a correção.

O AAP transforma o APM de uma ferramenta de observabilidade em uma plataforma unificada de observabilidade e segurança, sem a necessidade de agentes de segurança adicionais ou configurações complexas.

Tipos de Ameaças Detectadas em Tempo Real

O App and API Protection (AAP) do Datadog é projetado para detectar e proteger contra uma vasta gama de ameaças que visam aplicações web e APIs. Diferente de soluções de segurança mais genéricas, o AAP se beneficia do contexto

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog Observability Suite: APM, Watchdog, RUM, Synthetics & Security

de execução da aplicação (via APM) para identificar ataques com alta precisão e baixo índice de falsos positivos.

O AAP foca em ameaças que exploram vulnerabilidades comuns e comportamentos maliciosos, categorizadas da seguinte forma:

Vulnerabilidades OWASP Top 10: oferece detecção robusta para as vulnerabilidades mais críticas listadas no OWASP Top 10, que são frequentemente exploradas em aplicações .NET e outras tecnologias:

- A01:2021 - Broken Access Control: Detecção de tentativas de acesso a recursos ou funcionalidades sem a devida autorização.
- A03:2021 - Injection: Inclui Injeção SQL, XSS (Cross-Site Scripting), Command Injection e outras formas de injeção de código malicioso. O AAP identifica padrões de ataque em parâmetros de requisição, cabeçalhos e corpo.
- A04:2021 - Insecure Design: Embora o AAP não corrija falhas de design, ele pode detectar explorações de vulnerabilidades resultantes de um design inseguro, como a exposição de endpoints sensíveis.
- A05:2021 - Security Misconfiguration: Detecção de configurações inseguras que podem ser exploradas, como a exposição de diretórios ou arquivos de configuração.
- A06:2021 - Vulnerable and Outdated Components: Identificação de requisições que tentam explorar vulnerabilidades conhecidas (CVEs) em bibliotecas e frameworks de terceiros utilizados pela aplicação .NET.
- A07:2021 - Identification and Authentication Failures: Detecção de ataques de força bruta, enumeração de usuários e outras tentativas de comprometer credenciais.
- A08:2021 - Software and Data Integrity Failures: Detecção de manipulação de dados em trânsito ou em repouso.
- A09:2021 - Security Logging and Monitoring Failures: O AAP, ao gerar eventos de segurança, ajuda a preencher essa lacuna, fornecendo logs de segurança detalhados.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- A10:2021 - Server-Side Request Forgery (SSRF): Detecção de tentativas de forçar o servidor (aplicação .NET) a fazer requisições para recursos internos ou externos não autorizados.

API Abuse e Ataques à Lógica de Negócio: são alvos frequentes de ataques que não se encaixam nas categorias tradicionais de vulnerabilidades, mas que exploram a lógica de negócio da aplicação. O AAP é capaz de detectar:

- **Enumeração de Usuários**: tentativas de descobrir usuários válidos através de respostas de API (ex: user not found vs invalid password).
- **Rate Limiting Bypass**: tentativas de contornar limites de requisição para realizar ataques de força bruta ou DDoS de camada 7.
- **Fraude de Negócio**: detecção de padrões de requisição que indicam tentativas de fraude (ex: múltiplos pedidos de teste com cartões de crédito inválidos).

Detecção de Bots Maliciosos: pode identificar e mitigar o tráfego de bots maliciosos que realizam scraping, preenchimento de credenciais (credential stuffing) ou ataques de negação de serviço distribuído (DDoS) de camada de aplicação.

A detecção em tempo real e a capacidade de correlacionar esses eventos com o trace do APM fornecem às equipes de segurança e desenvolvimento a visibilidade necessária para entender, priorizar e responder a essas ameaças de forma eficaz.

Analisando o Trace da Requisição Maliciosa

Quando o App and API Protection (AAP) detecta uma ameaça, ele não apenas gera um alerta; ele enriquece o trace do APM com um nível de detalhe de segurança sem precedentes. O trace de uma requisição maliciosa se transforma em uma evidência forense completa e contextualizada, permitindo que as equipes de segurança e desenvolvimento compreendam o ataque em profundidade e respondam de forma eficaz.

A análise do trace de segurança no Datadog Security Explorer ou no Trace Explorer fornece um panorama detalhado do incidente, integrando informações de observabilidade e segurança.

Contexto da Requisição e do Atacante: o trace capturado pelo AAP inclui informações cruciais sobre a origem e a natureza da requisição maliciosa:

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack" APM, Watchdog, RUM, Synthetics & Security

- IP de Origem (http.client_ip): o endereço IP do cliente que originou a requisição, permitindo identificar a localização geográfica ou a rede do atacante.
- Payload (http.request.body, http.request.query, http.request.headers): o corpo completo da requisição, parâmetros de URL e cabeçalhos HTTP. O AAP destaca a string ou o padrão exato que acionou a regra de detecção, como um script XSS ou uma injeção SQL.
- Endpoint Alvo (http.url, http.method): o endpoint específico da aplicação .NET (ex: POST /api/users/login) que foi alvo do ataque, indicando a funcionalidade explorada.
- User Agent (http.useragent): informações sobre o navegador ou cliente utilizado pelo atacante, que podem revelar ferramentas de ataque ou bots.

Contexto da Aplicação: por ser integrado ao APM, fornece detalhes internos da aplicação que são vitais para a investigação:

- Serviço e Versão (service.name, version): o nome do serviço .NET (ex: ecommerce-api-dotnet) e a versão exata do código implantado. Isso é fundamental para determinar se a vulnerabilidade foi introduzida em um deploy específico ou se afeta outras versões.
- Usuário Autenticado (usr.id, usr.email, usr.name): se o ataque foi realizado por um usuário autenticado (ex: um funcionário mal-intencionado ou uma conta comprometida), o trace incluirá o ID, e-mail e nome do usuário, correlacionados via RUM ou APM.
- Stack Trace do Backend (error.stack): em caso de erro ou exceção causada pelo ataque, o trace incluirá a stack trace completa do código .NET, apontando para a linha exata onde a falha ocorreu.

Contexto da Ameaça (AAP): adiciona metadados de segurança específicos ao trace, detalhando a detecção:

- Regra de Detecção (appsec.event.rule.name, appsec.event.rule.id): o nome e o ID da regra do AAP que foi acionada (ex: SQL_INJECTION_ATTACK, xss_attack).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack" (APM, Watchdog, RUM, Synthetics & Security)

- Tipo de Ataque (appsec.attack.type): a classificação do ataque (ex: sql_injection, xss, ssrf).
- Ação do AAP (appsec.event.blocking_action): se o AAP foi configurado para bloquear o ataque, o trace indicará a ação de bloqueio tomada (ex: block).

Com todas essas informações consolidadas em um único trace, as equipes podem:

- Diagnosticar Rapidamente: identificar a causa raiz e o impacto do ataque sem a necessidade de correlacionar dados de múltiplas ferramentas.
- Priorizar Correções: focar nos endpoints e vulnerabilidades que estão sendo ativamente explorados em produção.
- Validar Defesas: confirmar se as correções implementadas ou as regras de bloqueio do AAP estão funcionando conforme o esperado.

O trace enriquecido pelo AAP é a espinha dorsal de uma resposta a incidentes de segurança ágil e baseada em dados, transformando a observabilidade em uma poderosa ferramenta de segurança.

Integração com CI/CD e SAST

A segurança de aplicações eficaz exige uma abordagem que permeie todo o ciclo de vida de desenvolvimento, desde a concepção até a produção. Este é o princípio do "Shift Left Security", onde vulnerabilidades são identificadas e corrigidas o mais cedo possível. O Datadog App and API Protection (AAP), embora opere em tempo de execução, integra-se de forma poderosa com o pipeline de CI/CD e complementa ferramentas de Análise Estática de Segurança de Aplicações (SAST), criando uma estratégia de segurança multicamadas para aplicações.

AAP como um Security Gate no CI/CD: embora o AAP seja uma solução de proteção em tempo de execução, ele pode influenciar diretamente o pipeline de CI/CD através da detecção de vulnerabilidades em dependências e da validação de segurança de builds.

- Análise de Composição de Software (SCA) Integrada: o Datadog oferece funcionalidades de SCA que podem escanear as dependências do seu projeto durante o build no pipeline de CI/CD. Ele identifica bibliotecas com vulnerabilidades conhecidas (CVEs).

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- Gates de Segurança: com base nos resultados do SCA, o pipeline pode ser configurado para falhar se vulnerabilidades críticas forem detectadas. Isso impede que builds contendo dependências inseguras sejam promovidos para ambientes de produção.
- Feedback Rápido: desenvolvedores recebem feedback imediato sobre vulnerabilidades de dependência, permitindo que atualizem as bibliotecas antes mesmo do deploy.

Complementaridade com SAST (Static Analysis Security Testing): ferramentas SAST (como SonarQube, Checkmarx, Snyk Code) analisam o código-fonte C# em busca de padrões de código inseguros, vulnerabilidades e bad practices antes da execução. O AAP e o SAST trabalham em conjunto para fornecer uma visão mais completa da postura de segurança:

- SAST: detecção de Vulnerabilidades Potenciais: O SAST identifica vulnerabilidades no código-fonte (ex: um método que constrói uma query SQL de forma insegura, um campo de entrada não sanitizado). Ele atua como uma primeira linha de defesa, encontrando problemas antes que a aplicação seja executada.
- AAP: validação e Priorização em Tempo Real: O AAP entra em ação na produção, confirmando se as vulnerabilidades potenciais identificadas pelo SAST estão sendo ativamente exploradas por atacantes reais. Se o SAST encontrar 100 vulnerabilidades, o AAP pode indicar quais delas estão sob ataque, permitindo que a equipe de desenvolvimento priorize a correção com base no risco real e no impacto em produção.

Fluxo de Desenvolvimento Seguro Otimizado:

- Desenvolvimento: desenvolvedores escrevem código.
- SAST (Pré-Commit/CI): o SAST analisa o código-fonte, identificando vulnerabilidades potenciais. Feedback é dado ao desenvolvedor.
- SCA (CI Build): o Datadog escaneia as dependências do projeto. Se vulnerabilidades críticas forem encontradas, o build falha.
- Deploy: o código seguro é implantado em produção.

FIAP
PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack" (APM, Watchdog, RUM, Synthetics & Security)

- AAP (Runtime): o AAP monitora a aplicação em tempo real, detectando e bloqueando ataques, e correlacionando-os com os traces do APM. Ele valida se as vulnerabilidades (potenciais ou não) estão sendo exploradas.

Essa integração garante que a segurança seja uma preocupação contínua, desde o código-fonte até a execução em produção, e que os esforços de correção sejam direcionados para as ameaças mais relevantes e de maior impacto

Como o AAP Reduz MTTR em Incidentes de Segurança

O Mean Time to Resolution (MTTR) é uma métrica fundamental para avaliar a eficiência de uma equipe na resposta a incidentes. No contexto de segurança, um MTTR baixo significa que as ameaças são detectadas, investigadas e corrigidas rapidamente, minimizando o impacto de um ataque. O MTTR em segurança pode ser decomposto em três componentes principais:

MTTR_{Segurança} = MTTA_{Detecção} + MTTI_{Investigação} + MTTK_{Correção}

O Datadog App and API Protection (AAP) foi projetado para impactar positivamente cada um desses componentes, transformando a resposta a incidentes de segurança de um processo manual e demorado em um fluxo de trabalho automatizado, contextualizado e ágil.

Redução do MTTA (Mean Time to Acknowledge/Detect): acelera a detecção e o reconhecimento de ameaças de várias maneiras:

- Detecção em Tempo Real: ao operar in-process com o Tracer do APM, o AAP detecta ataques no momento exato em que ocorrem, antes mesmo que a requisição maliciosa seja processada pela lógica de negócio da aplicação .NET. Isso elimina o atraso inerente a soluções baseadas em logs ou varreduras periódicas.
- Alertas Contextualizados: os alertas gerados pelo AAP já vêm enriquecidos com o trace completo do APM, o payload do ataque, o endpoint alvo e o serviço .NET afetado. Essa riqueza de contexto permite que o engenheiro de segurança ou de plantão reconheça a natureza e a gravidade do incidente imediatamente, sem a necessidade de investigar informações adicionais.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack" (APM, Watchdog, RUM, Synthetics & Security)

Redução do MTTI (Mean Time to Investigate): a fase de investigação é frequentemente a mais demorada em um incidente de segurança. O AAP simplifica e acelera essa etapa:

- Evidência Forense Completa no Trace: conforme detalhado no Subtópico 5.3, o trace do APM enriquecido pelo AAP atua como uma evidência forense completa. Ele consolida todas as informações relevantes (IP de origem, payload, endpoint, contexto do usuário, stack trace do backend) em um único local. Isso elimina a necessidade de correlacionar manualmente dados de diferentes sistemas (WAF, logs, APM), que é um processo propenso a erros e demorado.
- Contexto de Aplicação Profundo: o AAP fornece visibilidade sobre o código .NET em execução, permitindo que o investigador identifique exatamente qual parte da aplicação foi afetada e como o ataque se propagou, acelerando a identificação da causa raiz.

Redução do MTTK (Mean Time to Knowledge/Fix): uma vez que a causa raiz é identificada, o AAP ajuda a acelerar a correção:

- Priorização Baseada em Risco Real: ao correlacionar as vulnerabilidades detectadas (via SCA/SAST) com os ataques ativos em produção (via AAP), as equipes de desenvolviment podem priorizar a correção das vulnerabilidades que estão sendo ativamente exploradas, focando seus esforços onde o risco é maior.
- Validação Imediata da Correção: após a implementação de uma correção no código, o engenheiro pode re-executar a requisição maliciosa e verificar no Datadog se o AAP não dispara mais o alerta ou se a ação de bloqueio está funcionando. Essa validação em tempo real garante que a correção foi eficaz e reduz o tempo de ciclo de feedback.

O AAP transforma a postura de segurança de reativa para proativa e eficiente. Ao integrar segurança diretamente na observabilidade, ele capacita as equipes a responder a incidentes de segurança com a mesma agilidade e contexto que respondem a incidentes de performance, garantindo que as aplicações .NET permaneçam seguras e resilientes em um ambiente de ameaças em constante mudança.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Durante a aula, exploramos profundamente o papel do Watchdog como pilar central em uma estratégia de AIOps. Discutimos como essa ferramenta automatiza a detecção de anomalias, regressões e padrões de erro, permitindo que a equipe foque em problemas críticos. Para isso, aprendemos a interpretar corretamente uma Story, analisando seus elementos fundamentais: o contexto do incidente, a correlação entre os eventos e as ações necessárias. Um ponto crucial dessa discussão foi o desenvolvimento do senso crítico para diferenciar os momentos de total confiança na automação do Watchdog daqueles que exigem uma investigação manual detalhada.

Avançando para a perspectiva do usuário, abordamos a importância vital do RUM (Real User Monitoring) para eliminar os pontos cegos do backend, garantindo visibilidade total da jornada do cliente. Entendemos tecnicamente como ocorre a propagação de contexto de tracing através de headers, o que nos permite analisar traces distribuídos Full-Stack — desde a interação inicial no navegador até a consulta final no banco de dados.

Para a identificação de gargalos, demonstramos o uso de Flame Graphs unificados, que facilitam a visualização de latências em sistemas complexos. Por fim, conectamos a performance técnica ao negócio, correlacionando os Core Web Vitals (LCP, FID e CLS) com o desempenho do backend. Essa análise permite avaliar o impacto técnico sempre com foco na experiência real do usuário, garantindo que as otimizações reflitam diretamente na satisfação do cliente final.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

REFERÊNCIAS

DATADOG. **Connect RUM and Traces**. Disponível em: https://docs.datadoghq.com/tracing/other_telemetry/rum/. Acesso em: 14 jan. 2026.

DATADOG. **Real User Monitoring (RUM).** Disponível em: https://docs.datadoghq.com/real_user_monitoring/. Acesso em: 14 jan. 2026.

DATADOG. **Trace Context Propagation**. Disponível em: https://docs.datadoghq.com/tracing/trace_collection/trace_context_propagation/. Acesso em: 14 jan. 2026

DATADOG. **Watchdog**. Disponível em: https://docs.datadoghq.com/watchdog/. Acesso em: 14 jan. 2026

GOOGLE. **Core Web Vitals**. Disponível em: https://web.dev/vitals/. Acesso em: 14 jan. 2026.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Datadog - Observabilidade "Full-Stack"
APM, Infrastructure, Logs, Watchdog, RUM, Synthetics & Security)

PALAVRAS-CHAVE

Full-Stack, AIOps, Experiência do Usuário (UX).

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com



PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com